

1. Architecture goal

Build the smallest production-shaped system that demonstrates a reliable payment-recovery control loop.

Principle: AI proposes; deterministic policy disposes; controlled tools execute; verified payment events determine truth.

2. System architecture

Razorpay Webhook / Simulator

|

v

Webhook Gateway

- signature verification
- event ID deduplication
- fast acknowledgement

|

v

Event / Case Service

- normalize event
- create/update recovery case
- state machine

|

+-----+

|

v

Context Builder

- payment
- order
- customer history
- prior actions

|

v

AI Decision Service

- structured output
- diagnosis
- recommendation
- confidence

|

+-----+

v

Policy Engine

Policy / Merchant Config

- limits
- retry rules
- consent/opt-out
- action allowlist
- escalation thresholds

|

|

|

|

|

|

|

|

|

|

|

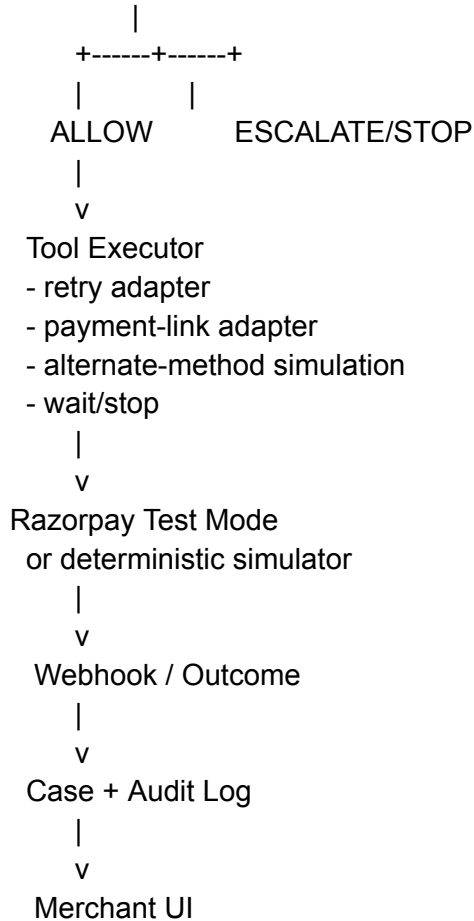
|

|

|

|

|



3. Recommended implementation shape

Prefer a single repository and a modular service architecture over microservices. Keep boundaries explicit in code so components can be split later without premature infrastructure.

Suggested modules:

- `webhooks/`
- `cases/`
- `context/`
- `agent/`
- `policy/`
- `tools/`
- `razorpay/`
- `simulator/`
- `evaluation/`
- `audit/`

- `ui/`

Use a relational database for authoritative state and audit records. PostgreSQL is the preferred production-shaped choice; SQLite is acceptable for local/demo execution if deployment constraints demand it.

4. Core domain model

RecoveryCase

- `id`
- `merchant_id`
- `event_id`
- `payment_id`
- `order_id`
- `payment_link_id`
- `amount`
- `currency`
- `failure_code`
- `failure_description`
- `payment_method`
- `customer_context`
- `attempt_count`
- `status`
- `recoverability_score`
- `expected_recovery_value`
- `consent_status`
- `policy_version`
- `created_at`
- `updated_at`

Decision

- `id`
- `case_id`
- `model_provider`
- `model_version`
- `prompt_version`
- `diagnosis`
- `evidence[]`

- recommended_action
- confidence
- expected_value
- created_at

PolicyCheck

- id
- decision_id
- allowed
- reasons[]
- policy_version
- created_at

ToolExecution

- id
- case_id
- decision_id
- tool_name
- idempotency_key
- arguments
- result
- status
- created_at

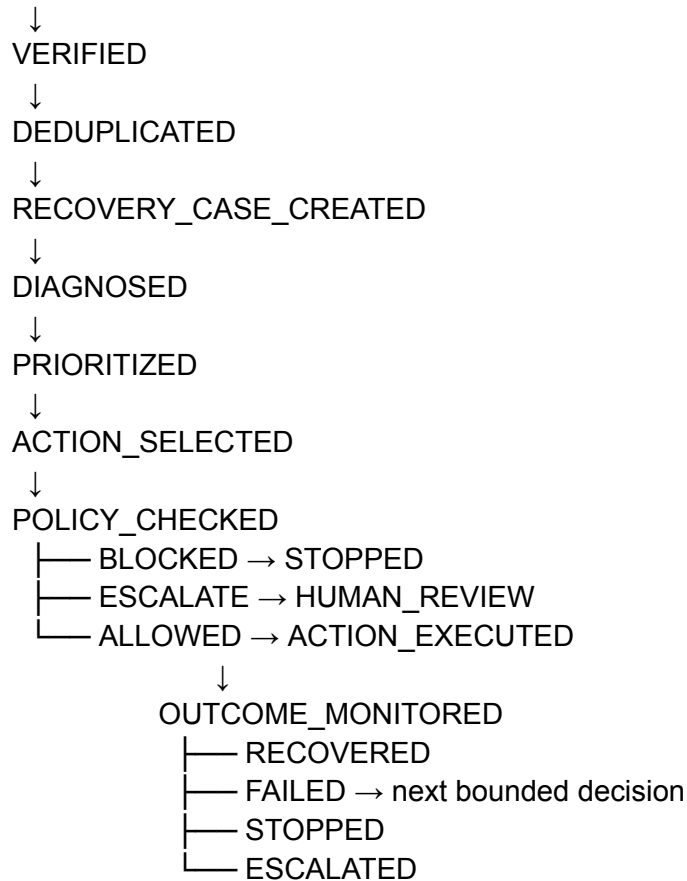
AuditEvent

Append-only event record containing:

- id
- case_id
- event_type
- actor
- source
- metadata
- timestamp

5. State machine

EVENT_RECEIVED



Every transition must be explicit and auditable.

6. LLM contract

The model receives a structured context object and must return only a typed JSON object matching a schema such as:

```
{
  "diagnosis": "transient_gateway_error",
  "evidence": ["..."],
  "recommended_action": "RETRY",
  "confidence": 0.87,
  "expected_recovery_value": 4200,
  "rationale": "..."
}
```

The model must never return executable API calls. Tool arguments are created/validated by application code.

If structured output is invalid, retry parsing within a bounded limit, then fall back to policy-only behavior or escalation.

7. Policy engine

Policy is deterministic and runs after the model.

Example checks:

1. Is the case still open?
2. Has payment already succeeded?
3. Is the customer opted out?
4. Is the recommended action allowed?
5. Is amount within autonomous threshold?
6. Has retry/action limit been reached?
7. Is this action valid for this failure category?
8. Is required evidence present?
9. Is the tool available and idempotency key unused?

Policy returns:

ALLOW | BLOCK | ESCALATE

No tool may execute without ALLOW.

8. Tool layer

Define a narrow internal interface:

- `retry_payment(case)`
- `create_or_reuse_payment_link(case)`
- `offer_alternate_method(case)`
- `wait_case(case)`
- `stop_case(case)`
- `escalate_case(case)`

Each tool must validate its own arguments and policy assumptions again before execution.

Use adapters:

- `RazorpayAdapter`
- `SimulatorAdapter`

This makes the demo resilient if a Test Mode capability is unavailable.

9. Webhook design

Webhook endpoint responsibilities:

1. Read raw payload.
2. Verify signature.
3. Extract event ID.
4. Check event ID/idempotency store.
5. Persist receipt.
6. Acknowledge quickly.
7. Process asynchronously.

A duplicate event should produce an audit entry but must not repeat the financial action.

10. Simulator design

Generate cases from hidden latent state:

`latent_state` → `observable_case` → `recoverability/action outcomes`

Example latent dimensions:

- transient vs persistent failure;
- customer willingness;
- payment-method viability;
- time sensitivity;
- prior attempt behavior;
- amount sensitivity.

The agent sees only observable fields. Outcomes are generated independently from hidden state.

Seed all benchmark runs.

11. Evaluation architecture

Evaluation runner executes the same cases through multiple strategies:

- `naive_baseline`
- `rule_baseline`
- `llm_only`

- `policy_only`
- `recoverai_hybrid`

Collect case-level and aggregate metrics. Persist benchmark configuration, seed, model version, policy version, and dataset version.

Never let the final held-out test set influence tuning.

12. Failure handling

LLM failure

Fallback to policy-only decisioning or escalation.

Tool failure

Persist the failed attempt, do not silently retry indefinitely, and escalate after bounded retries.

Webhook duplication

Ignore duplicate execution while preserving observability.

Out-of-order event

Reconcile against authoritative current payment state before executing another action.

Payment succeeds while action is pending

Cancel/skip further recovery actions and mark the case recovered.

Unknown failure code

Do not invent a recovery action. Use `ESCALATE` or `STOP`.

13. Security

- Store secrets in environment variables.
- Never commit Razorpay credentials.
- Never expose credentials to the LLM.
- Validate all external payloads.
- Log metadata without sensitive payment/customer secrets.
- Use least-privilege API credentials.
- Separate test credentials from production credentials.

14. Observability

Log:

- request/event IDs;
- case IDs;
- state transitions;
- model calls and latency;
- policy results;
- tool calls/results;
- webhook receipt and deduplication;
- final outcome.

Expose useful metrics in the UI without leaking secrets.

15. Deployment target

Optimize for a simple reproducible deployment. The exact provider is less important than:

- HTTPS webhook endpoint;
- persistent database;
- environment secrets;
- predictable startup;
- public demo URL if available.

Local simulator mode must remain usable if external credentials are unavailable.

16. Architecture decisions

ADR-01: Hybrid AI + deterministic policy

Decision: LLM handles contextual diagnosis/recommendation; deterministic policy controls money-affecting execution.

Reason: This preserves AI value while constraining financial risk.

ADR-02: Single agent

Decision: One recovery decision agent.

Reason: Multi-agent architecture adds failure modes without improving the core hiring signal.

ADR-03: Provider adapter

Decision: Separate Razorpay adapter from simulator.

Reason: Protects the end-to-end demo from API/test-mode uncertainty.

ADR-04: Event-sourced audit trail

Decision: Append-only audit events for critical transitions and tool actions.

Reason: Recovery decisions must be explainable and reproducible.